ITA06 – MACHINE LEARNING

Assignment 4 – CO4

Instance-Based Learning: k-NN, Locally Weighted Regression, RBF Networks, Case-Based Reasoning

Field	Detail
Name	M. Pranay
Register No	192525382

Q1 – k-Nearest Neighbour Classification (6 marks)

1.1 Why k-NN is a lazy (instance-based) learner

k-NN builds **no explicit model** during training. "Training" means only **storing** the labelled instances (and, optionally, fixing k, the distance metric and any feature scaling). All computation – distance calculation, ranking and voting – is postponed until a query arrives, and the target function is approximated **locally** around each query. Training cost is therefore O(1) (just storage), while prediction costs O(n·d) per query with brute force, and memory grows linearly with n. An eager learner (e.g. logistic regression) does the opposite: it spends time at training, compresses the data into a fixed parameter vector and answers queries cheaply.

1.2 Query (3, 3): Euclidean distance table

$d(P, q) = \sqrt{[(Vib - 3)^2 + (Temp - 3)^2]}$. The squared distance is exact; the root is rounded to 4 d.p.

ID	Vib	Temp	Class	d^2	d (Euclid)	Rank
P1	1	2	Normal	5	2.2361	6
P2	2	1	Normal	5	2.2361	7
P3	2	3	Normal	1	1.0000	2
P4	3	2.5	Worn	0.25	0.5000	1
P5	4	4	Worn	2	1.4142	3
P6	4.5	3	Worn	2.25	1.5000	4
P7	3.5	1	Normal	4.25	2.0616	5
P8	5	5	Worn	8	2.8284	8

Ranked neighbours (ties P1/P2 at d = 2.2361 broken by lower index): **P4 (Worn, 0.5000), P3 (Normal, 1.0000), P5 (Worn, 1.4142), P6 (Worn, 1.5000), P7 (Normal, 2.0616), P1 (Normal, 2.2361), P2 (Normal, 2.2361), P8 (Worn, 2.8284)**.

k	Metric	Neighbours used	Worn : Normal	Prediction
1	Euclid	P4	1 : 0	Worn
3	Euclid	P4, P3, P5	2 : 1	Worn
5	Euclid	P4, P3, P5, P6, P7	3 : 2	Worn
3	Manhattan	P4, P3, P6	2 : 1	Worn

Manhattan distance, k = 3

$d_1(P, q) = |Vib - 3| + |Temp - 3|$.

ID	Class	$\|\Delta Vib\|$	$\|\Delta Temp\|$	d (Manhattan)	Rank
P1	Normal	2.00	1.00	3.00	6
P2	Normal	1.00	2.00	3.00	7
P3	Normal	1.00	0.00	1.00	2
P4	Worn	0.00	0.50	0.50	1
P5	Worn	1.00	1.00	2.00	4
P6	Worn	1.50	0.00	1.50	3
P7	Normal	0.50	2.00	2.50	5
P8	Worn	2.00	2.00	4.00	8

Manhattan ranking: P4 (0.5, Worn), P3 (1.0, Normal), P6 (1.5, Worn) → 2 Worn : 1 Normal → **Worn**. The query is classified **Worn** for k = 1, 3, 5 (Euclid) and k = 3 (Manhattan); the result is stable here because the query lies well inside the Worn side of the boundary (Section 1.4).

1.3 A second query where the prediction changes with k

Chosen query **q_2 = (4.5, 1.25)**, found by scanning a 0.25-step grid for a point whose predicted class changes with k (an exhaustive scan was needed – such points exist only near the class border, close to P7/P6/P4).

Rank	ID	Class	Euclid distance
1	P7	Normal	1.0308
2	P6	Worn	1.7500
3	P4	Worn	1.9526
4	P2	Normal	2.5125
5	P5	Worn	2.7951
6	P3	Normal	3.0516
7	P1	Normal	3.5795
8	P8	Worn	3.7832

k	Neighbours	Worn : Normal	Prediction
1	P7	0 : 1	Normal
3	P7, P6, P4	2 : 1	Worn
5	P7, P6, P4, P2, P5	3 : 2	Worn
7	P7, P6, P4, P2, P5, P3, P1	3 : 4	Normal

What this reveals. The prediction flips Normal → Worn → Worn → Normal as k goes 1 → 3 → 5 → 7. k fixes the size of the neighbourhood that is trusted: k = 1 follows the single closest point (P7), k = 3–5 bring in the Worn points P6 and P4, and k = 7 uses 7 of the 8 stored points so the vote is almost the global class count (4 Normal : 4 Worn overall) and the locality of the method is lost. With n = 8 the usable range of k is tiny; in general k must be chosen on held-out data (validation / cross-validation), not by inspecting a single query. Odd k avoids vote ties in a two-class problem.

1.4 1-NN decision regions (Voronoi sketch)

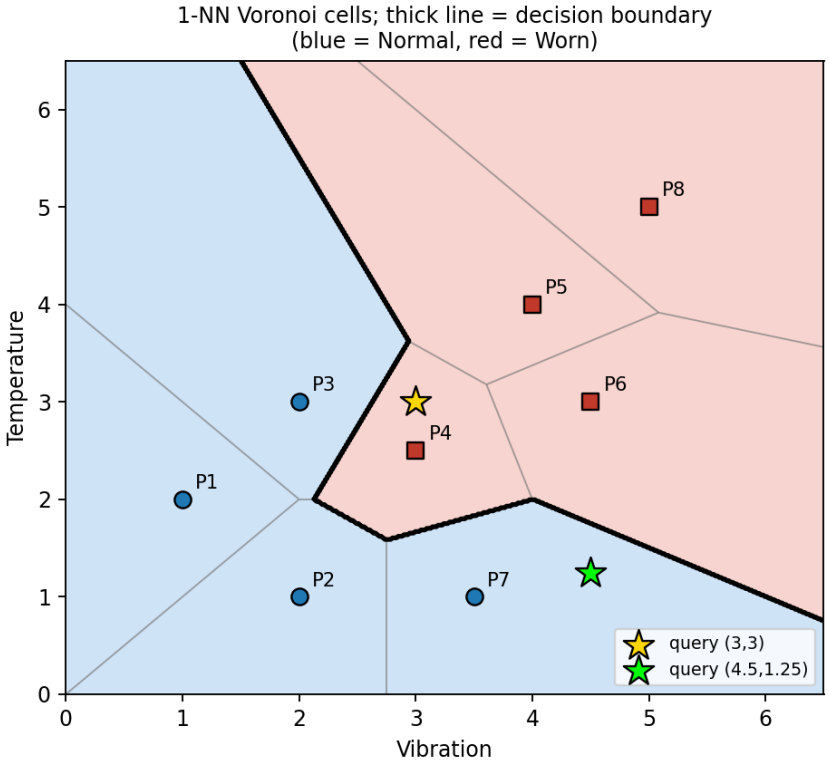


Figure 1.1 – Voronoi cells of the eight stored points (grey) and the 1-NN decision boundary (thick black). Both queries are shown as stars.

Each cell contains all points closer to its generator than to any other stored point; the cell edges are perpendicular bisectors of neighbouring points. The 1-NN decision boundary keeps only the edges that separate cells of **different** classes. It is piecewise linear and consists of the bisectors of the pairs (P3,P5) – through (3, 3.5) with slope −2 – then (P3,P4), (P2,P4), (P7,P4) and finally (P7,P6), which passes through (4, 2) with slope −½. Same-class edges (e.g. P1|P2, P5|P8) are removed. Query (3, 3) lies inside P4's Worn cell; q_2 (4.5, 1.25) lies inside P7's Normal cell, which is why 1-NN says Normal for it.

1.5 From-scratch implementation and library verification

Tie-breaking rule. (i) Equal distances: the stable sort keeps the point with the lower index first. (ii) Equal votes between classes (only possible for even k or weighted votes): the class of the **nearest** neighbour among the tied classes wins. (iii) Features are used as given (Q1 data are on one scale).

```
def knn_predict(Xtr, ytr, Xq, k, p=2, weighted=False, feat_w=None):
    Xq = np.atleast_2d(Xq); diff = Xq[:, None, :] - Xtr[None, :, :]
    if feat_w is not None: diff = diff * feat_w
    D = np.sqrt((diff**2).sum(-1)) if p == 2 else np.abs(diff).sum(-1)
    out = []
    for i in range(len(Xq)):
        idx = np.argsort(D[i], kind="stable")[:k]        # ties -> lower index
        w = 1/np.maximum(D[i, idx], 1e-12)**2 if weighted else np.ones(k)
        votes = {c: w[ytr[idx] == c].sum() for c in np.unique(ytr)}
        top = [c for c, s in votes.items() if np.isclose(s, max(votes.values()))]
        out.append(top[0] if len(top) == 1 else next(c for c in ytr[idx] if c in top))
    return np.array(out)
```

Verification against scikit-learn's KNeighborsClassifier (brute force) on the two queries plus 200 random points in $[0, 6]^2$:

Metric	k	Predictions identical	Test points
euclidean	1	202	202
euclidean	3	202	202

Metric	k	Predictions identical	Test points
euclidean	5	202	202
euclidean	7	202	202
manhattan	1	202	202
manhattan	3	202	202
manhattan	5	202	202
manhattan	7	202	202

All 8 settings agree on all 202 points, and the hand predictions above are reproduced by the code.

1.6 Reasoning inquiry – effect of k on the boundary

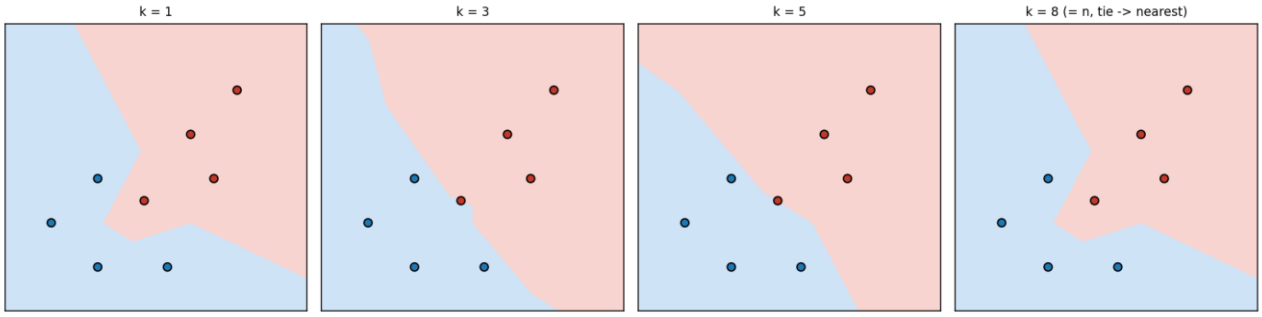


Figure 1.2 – 1-NN-to-k-NN decision regions on the Q1 data (k = 8 = n; the 4–4 vote tie is resolved by the nearest neighbour).

- **k = 1 (high variance, low bias):** the boundary is the Voronoi border, jagged and wrapped around every individual point; the training error is 0 (each point is its own neighbour) and a single mislabelled or shifted point moves the boundary locally.
- **k → n (high bias, low variance):** the neighbourhood is the whole training set, so every query receives the same global-majority label. The boundary disappears (all one colour; here the 4–4 tie makes even that arbitrary). The model ignores the inputs completely.
- Intermediate k gives a smoother boundary, trading variance for bias.

Evidence needed to confirm: (a) training and validation error as functions of k – expect training error rising from 0 and validation error U-shaped; (b) variance: refit on several bootstrap/resampled training sets and measure how often predictions at fixed test points change – large for k = 1, ~0 for k = n; (c) bias: compare the average prediction with the best achievable (Bayes) rate on a large held-out set – large for k = n; (d) behaviour when a fraction of training labels is flipped (done in Q6). With only 8 points these curves are too coarse, so the confirmation experiment is run on the larger datasets of Q2/Q6.

Q2 – Distance Weighting, Feature Scaling and the Curse of Dimensionality (6 marks)

2.1 Data and partitions

1,000 observations from make_classification (seed 42; 2 informative features, 2 clusters per class, 2% label noise). A monotone rescaling gives two features on very different scales: $\mathbf{x_1 = 5000 + 1500 \cdot z_1}$ (a count in the thousands) and $\mathbf{x_2 = sigmoid(z_2) \in (0, 1)}$ (a ratio). Stratified fixed split (seed 42): **600 train / 200 validation / 200 test** (train class-1 share 50.2%). Figure 2.1 shows the raw and standardised training data.

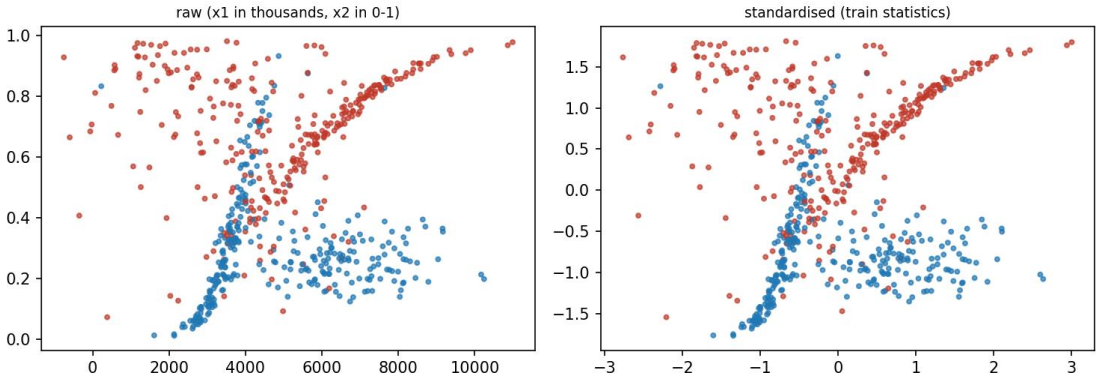


Figure 2.1 – Raw (left) and standardised (right) training data.

2.2 Predictions made before experimenting

- **(i) Standardisation** – large gain. Unscaled, x_1 (spread ≈ thousands) swamps x_2 (range < 1) in the Euclidean distance, so k-NN effectively uses one feature. Expect a jump of well over 10 accuracy points.
- **(ii) Weighting $w = 1/d^2$** – small effect at best. Weighting reduces the influence of far neighbours, so it should matter only when k is large; it also makes the prediction hypersensitive to very close (possibly mislabelled) points, so with 2% label noise it may even hurt slightly.
- **(iii) Irrelevant features** – accuracy falls steadily with each added noise feature because each adds unit variance to the squared distance, and the ratio d_nearest/d_farthest rises towards 1 (all points look equally far).

2.3 Unscaled vs standardised k-NN; selecting k

The scaler (mean, standard deviation) is **fitted on the 600 training rows only** and then applied to validation/test rows. k is chosen by validation accuracy (ties → smaller k).

k	Unscaled val. acc.	Standardised val. acc.
1	0.560	0.890
3	0.535	0.935
5	0.580	0.930
7	0.590	0.930
9	**0.590**	**0.940**
15	0.625	0.925
25	0.615	0.920

Unscaled best: k = 15 (0.625). **Standardised best: k = 9 (0.940)** – a gain of 31.5 points, confirming prediction (i). Unscaled accuracy stays between 0.535 and 0.625 for every k, because the ratio feature x_2 almost never influences which points are neighbours.

2.4 Unweighted vs distance-weighted voting (k = 9, five seeds)

Each seed s ∈ {1,…,5} regenerates the dataset and its stratified 600/200/200 split; k = 9 (chosen above) is held fixed; scaler fitted on that seed's training part.

Seed	Unscaled	Standardised, uniform	Standardised, w = 1/d²
1	0.540	0.830	0.800
2	0.850	0.865	0.820
3	0.500	0.960	0.960
4	0.870	0.885	0.895
5	0.600	0.915	0.895
mean ± sd	0.672 ± 0.175	0.891 ± 0.049	0.874 ± 0.065

On the main partition the validation accuracies are uniform 0.940 vs weighted 0.920. Weighting gives **no gain** (mean difference ≈ -1.7 points, smaller than the seed-to-seed standard deviation of ≈ 5–6 points), consistent with prediction (ii). Standardisation, in contrast, lifts the mean from 0.672 to 0.891 and also removes most of the seed-to-seed variance of the unscaled model.

2.5 Irrelevant features and distance concentration

0, 2, 5, 10, 20, 50 independent N(0,1) noise features are appended to the main partition. For every setting the scaler is refitted on the (augmented) training data, k = 9, uniform vote; three noise seeds (11, 12, 13). The contrast ratio is the mean over validation queries of d_nearest / d_farthest to the training set.

Features	Irrelevant	Val. accuracy (mean ± sd)	d_nearest / d_farthest
2	0	0.940 ± 0.000	0.016
4	2	0.905 ± 0.017	0.083
7	5	0.867 ± 0.026	0.188
12	10	0.833 ± 0.013	0.311
22	20	0.763 ± 0.008	0.426
52	50	0.723 ± 0.050	0.589

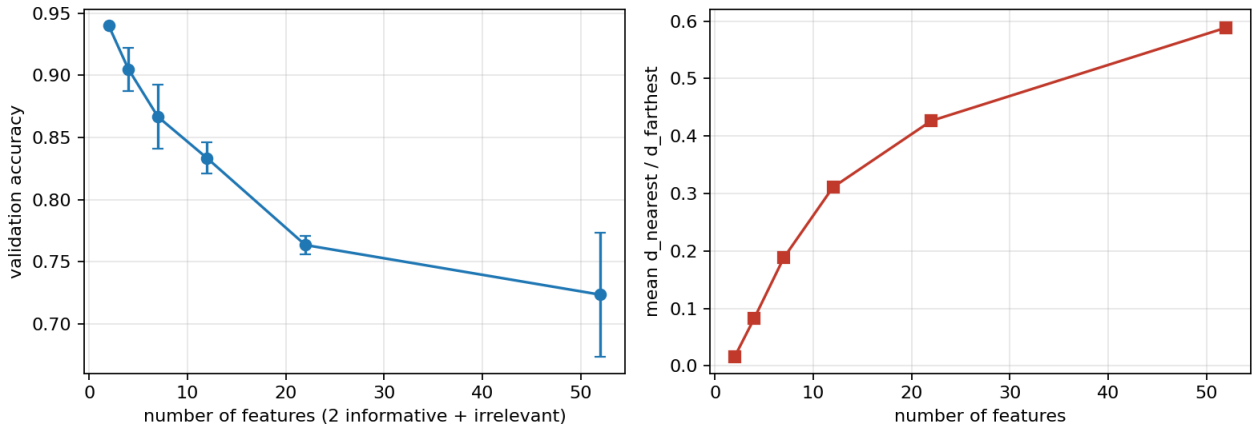


Figure 2.2 – Validation accuracy (left) and nearest/farthest distance ratio (right) vs number of features.

2.6 Final configuration and test result

Chosen configuration: **standardised features (2 informative), k = 9, unweighted (majority) vote**, because it had the best validation accuracy and weighting did not help. Fitted once on train + validation (scaler refitted on those 800 rows) and evaluated **once** on the untouched test partition: **test accuracy = 0.930** (200 samples → standard error ≈ 1.8 points).

2.7 Analysis inquiry

Why distances lose information as dimensionality grows. For a point with F coordinates the squared distance is a sum of F per-feature terms. Irrelevant features add a random, label-independent term (expected value 2 for two N(0,1) draws) that grows with F, while the informative part stays fixed; so all pairwise distances concentrate around the same value and the relative contrast d_nearest/d_farthest tends to 1 (here 0.016 → 0.589 from 2 to 52 features). The nearest neighbour is then chosen largely by noise, and the accuracy falls from 0.94 to 0.72.

Remedies. (1) **Feature selection** is the most direct fix when some features are known to be useless – it removes the noise terms entirely (tested in Q7: accuracy returns to 0.94 at 52 features). (2) **Feature weighting** (relevance-weighted distance) is the soft version and keeps weak features, but needs a good relevance estimate (Q7 shows a marginal-MI estimate can be misleading when features interact). (3) **Dimensionality reduction** (PCA) is the least suitable here: PCA keeps directions of large variance, and after standardisation the noise features have the same variance as the informative ones, so PCA cannot separate them; a supervised projection (LDA/PLS) would be needed. Evidence: Figure 2.2 shows accuracy decaying monotonically with irrelevant features and the contrast ratio rising – the cure must remove or down-weight those coordinates rather than re-tune k (Q7 also shows that re-tuning k recovers only part of the loss).

Why the test set must stay out of the choice of k. Choosing k (or the scaling/weighting scheme) by test accuracy lets the selection exploit sampling noise of that particular set, so the reported number becomes an optimistically biased estimate – the test set is no longer an independent measurement of generalisation. Likewise the scaler is fitted on training data only; fitting it on all data would leak the validation/test mean and variance (data leakage). k is therefore selected on the validation partition and the test partition is used exactly once for the final, already-fixed configuration.

Q3 – Locally Weighted Regression (7 marks)

3.1 Model and derivation

For a query x_q, locally weighted linear regression fits a straight line **only for that query**, giving each training point a Gaussian-kernel weight that decays with its distance from x_q:

$$w_i = \exp(-(x_i - x_q)^2 / (2\tau^2)), \quad \hat{y}(x_q) = \theta^T[1, x_q]$$

The parameters minimise the weighted sum of squared errors, with $x_i = [1, x_i]^T$ the augmented input:

$$J(\theta) = \Sigma_i w_i (y_i - \theta^T x_i)^2 = (y - X\theta)^T W (y - X\theta), \quad W = \mathrm{diag}(w_1,\ldots,w_n)$$

Expanding and differentiating (W is symmetric):

$$\nabla\theta J = -2 X^T W(y - X\theta) = -2 X^T Wy + 2 X^T WX\theta$$

Setting the gradient to zero gives the weighted normal equations $X^T WX\,\theta = X^T Wy$, hence

$$\theta = (X^T WX)^{-1} X^T Wy$$

The Hessian $2X^T WX$ is positive definite whenever at least two distinct x-values have non-zero weight, so this is the unique minimum. The prediction is $\hat{y}(x_q) = \theta^T[1, x_q]$. Because the weights depend on x_q, **a new θ is solved for every query**; with all $w_i = 1$ the method reduces to ordinary least squares.

3.2 Hand calculation: x_q = 3.5, τ = 1

x	y	$(x - 3.5)^2$	$-(x - 3.5)^2/2$	w = exp(·)
1	1.2	6.25	-3.125	0.0439
2	1.9	2.25	-1.125	0.3247
3	3.2	0.25	-0.125	0.8825
4	3.9	0.25	-0.125	0.8825
5	5.5	2.25	-1.125	0.3247

$\Sigma w = 2.4582$, $\Sigma w\cdot x = 8.4940$, $\Sigma w\cdot x^2 = 31.5213$, $\Sigma w\cdot y = 8.7209$, $\Sigma w\cdot x\cdot y = 32.4533$.

$$X^T WX = [[2.4582, 8.4940], [8.4940, 31.5213]], \quad X^T Wy = [8.7209, 32.4533]^T$$

$$\det(X^T WX) = 5.3390, \quad (X^T WX)^{-1} = (1/5.3390) \cdot [[31.5213, -8.4940], [-8.4940, 2.4582]]$$

$$\theta = (X^T WX)^{-1} X^T Wy = [-0.1431, 1.0681]^T$$

Prediction: $\hat{y}(3.5) = -0.1431 + 1.0681 \times 3.5 =$ **3.5953**. (The coded implementation returns 3.5953, which verifies the hand result.)

Global ordinary least squares on the same five points: $X^T X = [[5, 15], [15, 55]]$, $X^T y = [15.7, 57.7]^T \rightarrow \theta = [-0.040, 1.060]^T$, so $\hat{y}(3.5) = -0.040 + 1.060 \times 3.5 =$ **3.670**. The local prediction is 0.075 lower: the weights concentrate on x = 3 and 4 (w = 0.88 each), whose y-values (3.2, 3.9) average 3.55 and so lie below the global line's value of 3.67 at 3.5, whereas the global fit is also pulled by the distant points x = 1 and x = 5 (y = 5.5). The five points are nearly linear, so the two predictions are close; the advantage of LWR appears when the relationship bends (Section 3.4).

3.3 Implementation

```
def lwr_predict(x, y, xq_arr, tau, ridge=1e-8):
    out = np.empty(len(xq_arr))
    for i, xq in enumerate(xq_arr):
        d2 = (x - xq)**2
        w  = np.exp(-(d2 - d2.min())/(2*tau**2))     # constant rescaling of W leaves theta unchanged
        X  = np.c_[np.ones_like(x), x - xq]          # centred on xq: prediction = theta[0]
        A  = X.T @ (w[:, None]*X) + np.diag([0.0, ridge])
        out[i] = np.linalg.solve(A, X.T @ (w*y))[0]
    return out
```

Two numerical safeguards, neither of which changes the mathematics: the features are centred on x_q (so $\hat{y} = \theta_0$ and the intercept is the prediction), and the weights are rescaled by their maximum ($\exp(-(d^2 - d^2_{min})/2\tau^2)$) to avoid underflow when τ is tiny; a 10^{-8} ridge on the slope keeps X^TWX invertible when one point carries all the weight.

3.4 Experiments

Data: 100 points, x ~ U(0, 10) (seed 7), y = sin x + N(0, 0.2^2). Random split (seed 8): 70 training / 30 validation. The irreducible error (noise variance) is 0.04. LWR training error is computed by predicting the training points from the training set.

τ	Training MSE	Validation MSE
0.05	0.0053	0.1066
0.2	0.0230	0.0319
0.5	0.0340	0.0306
1	0.0864	0.0756
3	0.3621	0.3409
100	0.4768	0.4229

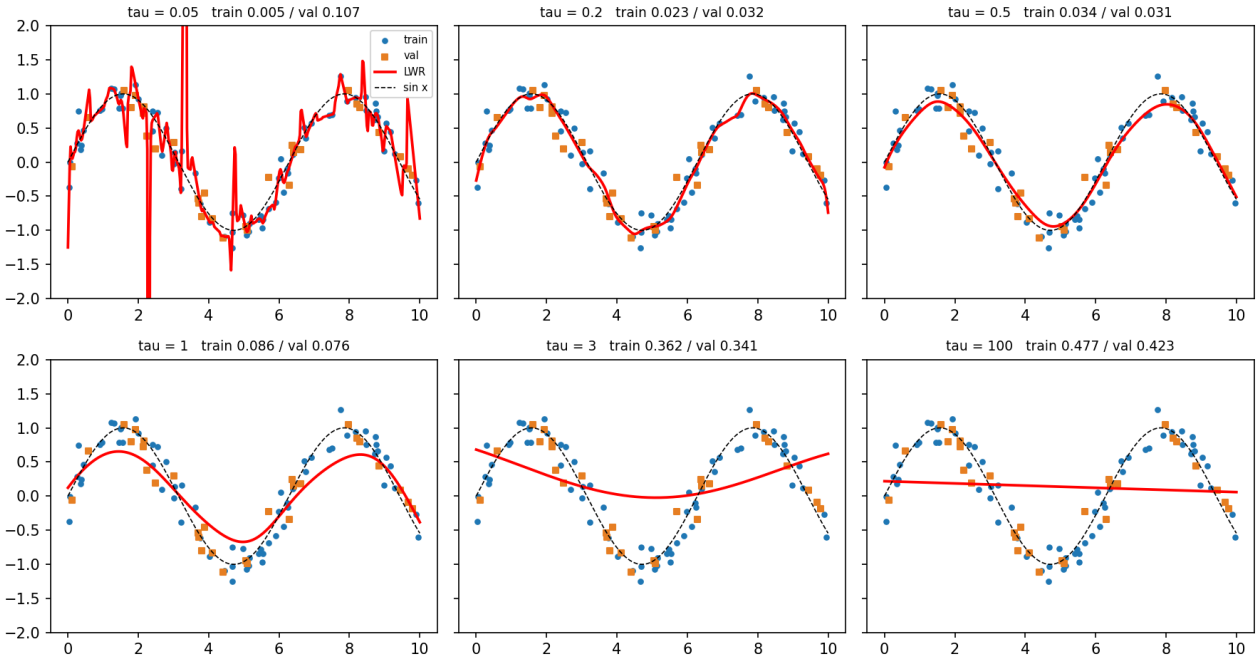


Figure 3.1 – LWR fits for six bandwidths (red) against sin x (dashed). Blue = training, orange squares = validation.

Comparison with global models

Model	Training MSE	Validation MSE	Predict 100 queries (ms)	Fit time (ms)
LWR, τ = 0.5	0.0340	0.0306	2.318	0 (lazy)
Global linear	0.4770	0.4230	0.0038	0.020
Polynomial deg 3	0.3623	0.3417	≈ 0.0072	≈ 0.030
Polynomial deg 5	0.0523	0.0349	0.0072	0.030
Polynomial deg 9	0.0276	0.0286	≈ 0.0072	≈ 0.030

Timing: mean of repeated runs on one CPU core (LWR: 20 repeats; polynomials: 2,000 repeats). Degree 3 and 9 prediction/fit times are of the same order as degree 5 and were not timed separately.

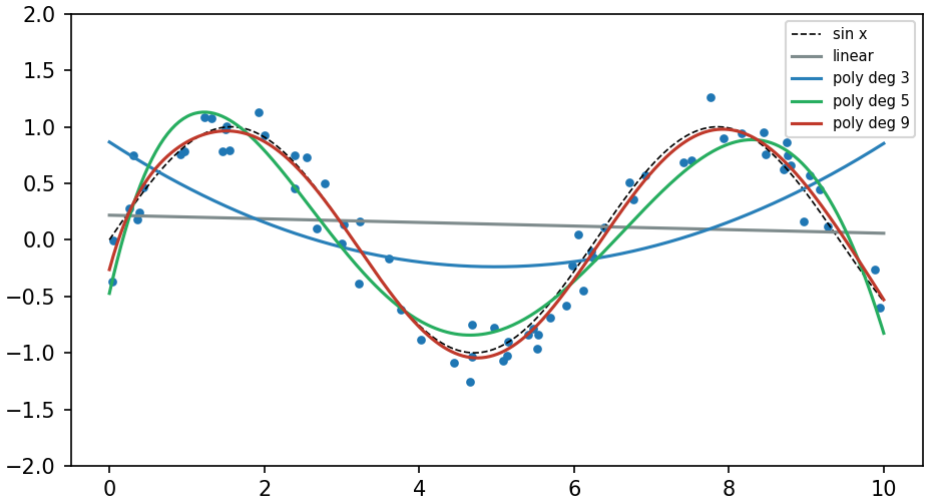


Figure 3.2 – Global linear and polynomial fits.

Results: a global line cannot follow the sine (validation MSE 0.423); cubic is still too stiff (0.342); degree 5 (0.035) and 9 (0.029) are competitive with LWR at τ = 0.2–0.5 (0.032, 0.031). LWR gives this accuracy without choosing a global functional form, but it is about **320× slower** at prediction (2.32 ms vs 0.0072 ms for 100 queries).

3.5 Validation inquiry

- **τ → 0:** the kernel is non-zero only for the nearest training point, so ŷ(x_q) → y of that point: LWR degenerates to **1-nearest-neighbour regression** (a staircase). The training error → 0 (Table: 0.0053 at τ = 0.05) while validation error rises (0.1066): **high variance / low bias**, the model fits the noise.
- **τ → ∞:** all weights → 1, so LWR becomes **ordinary global linear regression** (τ = 100: training MSE 0.4768, identical to the global line's 0.4770): **high bias / low variance**, underfitting the sine.
- A U-shaped validation curve with the minimum at τ ≈ 0.2–0.5 (≈ 0.031) is the bias–variance optimum; τ = 1 already over-smooths the peaks and τ = 3 is almost linear.

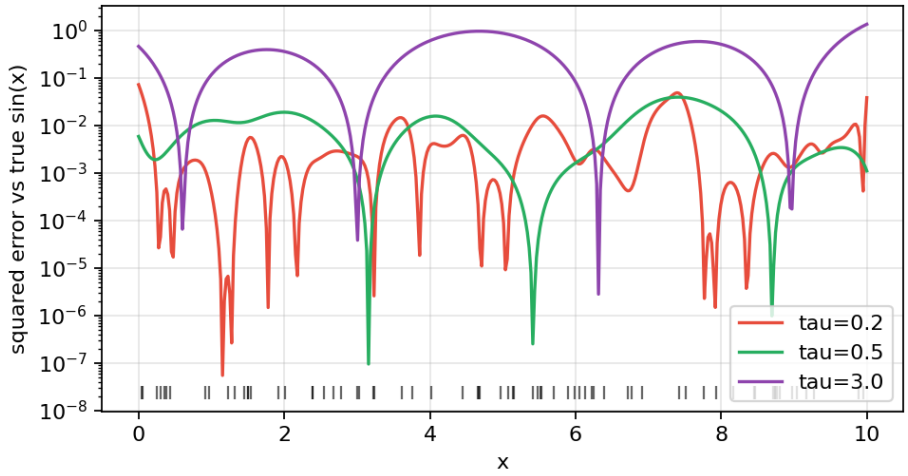


Figure 3.3 – Squared error against the true sin x along the input range (log scale; tick marks = training x-values).

Where predictions are least reliable. For small and medium τ the error rises sharply at the **two ends of the range** (x < 0.5 and x > 9.5: errors of 0.07 and 0.04 at τ = 0.2 against ≈ 10^{-3} in the interior) because all neighbours lie on one side, so the local line must extrapolate, and in **sparse stretches** – e.g. the largest gap between training x-values (0.61 wide, x = 9.28 to 9.89) just before the right end, and x ≈ 7.4 where the τ = 0.2 error peaks (≈ 0.05). Large τ (3.0) is unreliable everywhere the function curves (errors up to ≈ 1 near the maxima/minima of the sine). Note that with only five of the 30 validation points near the ends, the validation-set split by region is inconclusive (edge MSE 0.015 vs 0.034 at τ = 0.5), so the dense error-vs-position curve is the stronger evidence.

No fixed parameter set. LWR has no learned parameter vector: θ depends on x_q and is recomputed for every query from all stored points, so “training” is just storing the data (fit time 0). Prediction cost is therefore $O(n \cdot d^2)$ (weights plus a small d×d solve) **per query**, and memory is O(n); a polynomial or linear model, by contrast, predicts in O(d) with a constant-size parameter vector. This is the measured ~300–600× prediction-time gap above, and it grows with n.

Q4 – Radial Basis Function Network (7 marks)

4.1 Model

$$\phi_j(x) = \exp\left(-\|x - c_j\|^2 / (2\sigma^2)\right), \quad f(x) = \Sigma_j\, w_j\, \phi_j(x) + b$$

- **Centres c_j** – prototypes in input space; each hidden unit responds most strongly to inputs near its centre (they play the role of the stored instances in k-NN, but there are only $m \ll n$ of them).
- **Width σ** – the radius of influence of each unit. Small σ → very local, spiky bumps (may not cover the space, over-fit); large σ → heavily overlapping bumps, almost linear/global behaviour (under-fit).
- **Output weights w, b** – a linear layer on the hidden features; they are the only parameters fitted by least squares (or logistic regression). A nonlinear problem in x becomes a **linear** problem in ϕ-space (Cover's theorem).

4.2 XOR with two centres ($c_1 = (0,0)$, $c_2 = (1,1)$, $\sigma^2 = 0.5$)

With $2\sigma^2 = 1$: $\phi_1 = \exp(-\|x - c_1\|^2)$, $\phi_2 = \exp(-\|x - c_2\|^2)$.

Input	$\|x-c_1\|^2$	$\|x-c_2\|^2$	ϕ_1	ϕ_2	$\phi_1 + \phi_2$	XOR
(0,0)	0	2	1.0000	0.1353	1.1353	0
(0,1)	1	1	0.3679	0.3679	0.7358	1
(1,0)	1	1	0.3679	0.3679	0.7358	1
(1,1)	2	0	0.1353	1.0000	1.1353	0

Geometric argument. In (ϕ_1, ϕ_2) space the two XOR = 0 inputs map to the corners (1, 0.1353) and (0.1353, 1), while both XOR = 1 inputs collapse onto the **single** point (0.3679, 0.3679), which lies inside the segment between the XOR = 0 points (below the line $\phi_1 + \phi_2 = 1.1353$). The class-1 points therefore sit on one side of a straight line and the class-0 points on the other. A separating line is $\boldsymbol{\phi_1 + \phi_2 = 0.9}$ (class 1 when $\phi_1 + \phi_2 < 0.9$: $0.7358 < 0.9 < 1.1353$). The original XOR is not linearly separable in (x_1, x_2); in ϕ-space it is.

Least-squares output layer

Design matrix $\tilde{\Phi} = [\phi_1, \phi_2, 1]$ (rows in the order (0,0), (0,1), (1,0), (1,1)) and target $t = [0, 1, 1, 0]^T$:

$$\tilde{\Phi} = [[1.0000, 0.1353, 1], [0.3679, 0.3679, 1], [0.3679, 0.3679, 1], [0.1353, 1.0000, 1]]$$

$$\tilde{\Phi}^T\tilde{\Phi} = [[1.2890, 0.5413, 1.8711], [0.5413, 1.2890, 1.8711], [1.8711, 1.8711, 4.0000]], \quad \tilde{\Phi}^T t = [0.7358, 0.7358, 2.0000]^T$$

$$[w_1, w_2, b]^T = (\tilde{\Phi}^T\tilde{\Phi})^{-1}\,\tilde{\Phi}^T t = [-2.5027, -2.5027, 2.8413]^T$$

By symmetry $w_1 = w_2 = w$, and the system reduces to $w \cdot 1.1353 + b = 0$ and $w \cdot 0.7358 + b = 1$, i.e. $w = -1/0.3995 = -2.5027$, $b = 2.8413$. Predictions: $f(0,0) = -0.000 \approx 0$, $f(0,1) = 1.000$, $f(1,0) = 1.000$, $f(1,1) = 0.000 \approx 0$ – the four targets are reproduced **exactly** (residual $\approx 10^{-15}$). Thresholding at 0.5 gives the XOR truth table.

4.3 RBF network on a crescent dataset

Data: make_moons, 1,000 points, noise 0.25, seed 42; stratified split 600/200/200 (seed 42); features standardised with training statistics. **Centres:** k-means (k = m, 10 restarts, seed 0) on the **training data only**. **Width:** a shared σ, either from the sweep grid $\sigma \in \{0.05, 0.1, 0.25, 0.5, 1, 2\}$ or from the stated rule $\sigma = d_{max}/\sqrt{(2m)}$ (d_{max} = largest distance between centres). **Output layer:** least squares on $[\Phi, 1]$ with 0/1 targets (own NumPy code), prediction = 1 if $f(x) > 0.5$. **Parameters** = 2m (centres) + 1 (σ) + (m + 1) (weights, bias) = 3m + 2, of which m + 1 are fitted by least squares.

```
def rbf_fit(Xtr, ytr, m, sigma):
    C = KMeans(n_clusters=m, n_init=10, random_state=0).fit(Xtr).cluster_centers_   # train only
    H = lambda Z: np.exp(-((Z[:, None, :] - C[None])**2).sum(-1)/(2*sigma**2))
    w, *_ = np.linalg.lstsq(np.c_[H(Xtr), np.ones(len(Xtr))], ytr.astype(float), rcond=None)
    return lambda Z: (np.c_[H(Z), np.ones(len(Z))] @ w > 0.5).astype(int)
```

Training / validation accuracy (rows: number of centres m; columns: σ):

m	Params	σ=0.05	σ=0.1	σ=0.25	σ=0.5	σ=1	σ=2
2	8	0.51 / 0.50	0.52 / 0.51	0.59 / 0.59	0.83 / 0.79	0.83 / 0.81	0.84 / 0.81
5	17	0.53 / 0.51	0.57 / 0.56	0.77 / 0.73	0.82 / 0.82	0.85 / 0.83	0.85 / 0.81
10	32	0.59 / 0.58	0.75 / 0.77	0.90 / 0.87	0.94 / 0.92	0.94 / 0.91	0.92 / 0.88
20	**62**	**0.64 / 0.59**	**0.81 / 0.76**	**0.93 / 0.93**	**0.95 / 0.92**	**0.95 / 0.91**	**0.94 / 0.90**
50	152	0.76 / 0.65	0.93 / 0.86	0.95 / 0.91	0.96 / 0.92	0.95 / 0.92	0.95 / 0.92

m	σ from rule d_max/√(2m)	Train acc.	Val acc.
2	1.030	0.835	0.815
5	0.950	0.852	0.835
10	0.752	0.943	0.915
20	0.587	0.952	0.920
50	0.432	0.962	0.915

Best by validation: m = 20, σ = 0.25 (train 0.932, val 0.930, 62 parameters). **Under-fitted example:** m = 2, σ = 1 (train 0.835 / val 0.810, 8 parameters) – two Gaussians cannot follow a crescent. **Over-fitted example:** m = 50, σ = 0.1 (train 0.928 / val 0.865, 152 parameters) – many narrow bumps, 6.3-point train–validation gap.

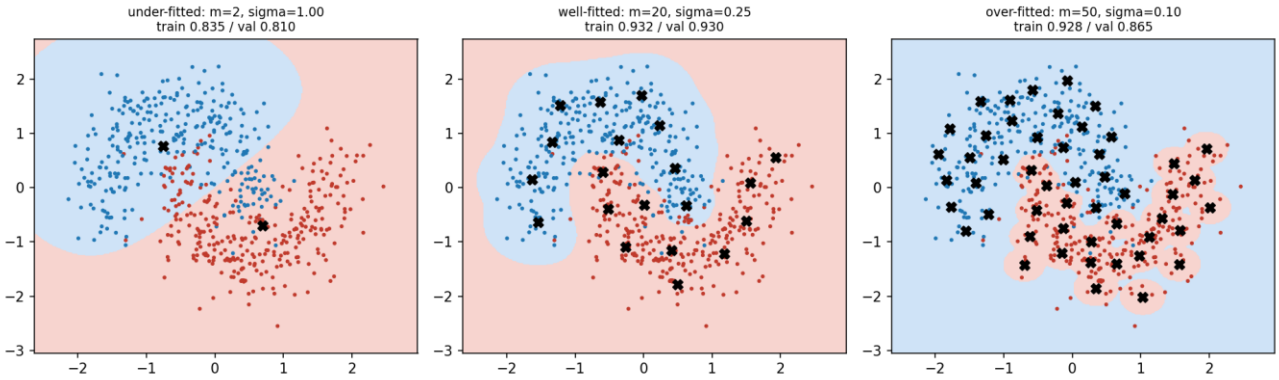


Figure 4.1 – Decision boundaries (black X = k-means centres). Left: under-fitted; centre: well-fitted; right: over-fitted (isolated islands around training clusters, and the background defaults to one class where no unit is active).

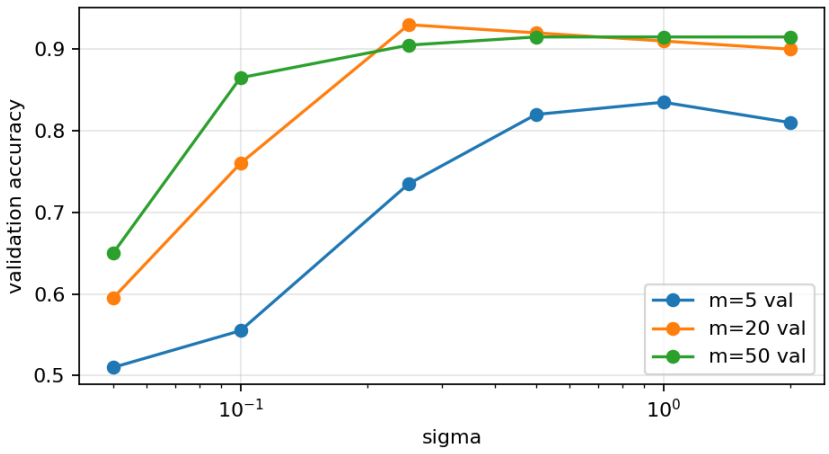


Figure 4.2 – Validation accuracy vs σ for m = 5, 20, 50.

4.4 Comparison with the best k-NN (after validation-based selection)

The Q2 k-NN procedure (standardise on train; k ∈ {1,3,5,7,9,15,25}; uniform or 1/d² vote chosen on validation) was re-applied to the **same crescent partitions** (the Q2 dataset has different features): best k-NN = k = 7, uniform,

validation accuracy 0.925. Both models are then refitted on train + validation and evaluated once on the test partition.

Model (selected on validation)	Validation acc.	Test acc. (200 pts)	Stored numbers
RBF network, m = 20, σ = 0.25	0.930	0.950	62
k-NN, k = 7	0.925	0.980	1600 (800 points × 2)

The validation accuracies are practically equal (0.930 vs 0.925). On test, k-NN is 3 points higher (0.98 vs 0.95, i.e. 6 test samples); with 200 test samples the standard error is ≈ 1.5 points, so this difference is **not statistically convincing**. What is clear is the cost: the RBF network keeps 62 numbers instead of 1,600 (≈ 26× fewer) and predicts with 20 kernel evaluations instead of 800 distance computations. On the Q2 dataset (two clusters per class) the same RBF procedure selected m = 10, σ = 1 (val 0.940) but reached only **0.840** test accuracy against 0.93 for k-NN: its training accuracy (0.860) was below its validation accuracy, a sign that a 0/1 least-squares output on 10 centres under-fits that layout and that the validation set was unusually easy.

4.5 Design inquiry

- **RBF vs k-NN:** k-NN stores all n instances and defers everything to prediction (O(n) per query, memory O(n)); an RBF network compresses the data into m prototypes, trains once, and predicts in O(m). k-NN adapts its local resolution automatically (the k nearest points); the RBF network has one global width, so regions of different density are treated alike.
- **RBF vs MLP:** an MLP learns hidden-layer weights by gradient descent (non-convex, may stall in local minima, needs a learning rate and epochs); in an RBF network the hidden layer is fixed by unsupervised steps and only a linear output layer is trained, which is a convex problem with a closed-form solution. The price is that centres are chosen without looking at the labels and the Gaussian units are local (poor extrapolation, many units needed in high dimensions).
- **Chosen without gradient descent:** the centres (k-means, an EM-like iteration), the width (a rule/validation grid) and the output weights (closed-form least squares).
- **Sensitivity to σ (observed):** very strong. At m = 20 the validation accuracy is 0.60 for σ = 0.05, 0.76 for σ = 0.1, **0.93 at 0.25** and 0.90 at σ = 2 (Fig. 4.2); too-small σ leaves gaps between centres (units almost never active), too-large σ blurs the crescent into a nearly linear boundary. The best σ moves with m – smaller for more centres – which is what the rule $d_{max}/\sqrt{2m}$ tries to capture (it gave 0.92 at m = 10 and 20).
- **When RBF beats storing all instances:** when n is large and memory or latency matter (streaming, embedded), when the data fall into compact clusters well described by few prototypes, and when a smooth closed-form model is preferred. When data are small, clusters are irregular or very noisy, or incremental updates are needed, storing the instances (k-NN) remains simpler.

Q5 – Case-Based Reasoning (5 marks)

5.1 The CBR cycle and its difference from k-NN

- **Retrieve** the stored case(s) most similar to the new problem; **Reuse** their solution (copy or adapt it); **Revise** the proposed solution against the new situation (expert check, simulation or feedback) and repair it if it does not fit; **Retain** the confirmed problem–solution pair as a new case (after checking it is useful and not redundant).

Difference from k-NN on numeric vectors. k-NN uses a fixed, purely numeric distance and returns a vote over class labels. CBR cases are structured, mixed-type records (symbolic, ordered, numeric) with a **tailored, weighted similarity**, the output is a complete solution (not just a label) that is usually **adapted** (revise) rather than copied, and the case base **learns** by retaining and also forgetting cases. Retrieval is typically for the single best (or a few best) cases, not a majority vote.

5.2 Similarity measure and weights

Global similarity (weighted average of local similarities, in [0, 1]):

$$\mathrm{Sim}(q, c) = \Sigma_a\, w_a \cdot \mathrm{sim}_a(q_a, c_a) \,/\, \Sigma_a\, w_a$$

- **Noise, Overheating (exact-match):** sim = 1 if the values are equal, else 0.
- **Pressure (ordered Low < Normal < High):** map to 0, 1, 2 and use sim = 1 − |rank(q) − rank(c)| / 2, so Low–Normal = 0.5 and Low–High = 0.
- **Weights: Noise 0.3, Overheating 0.4, Pressure 0.3.** Justification: overheating is the strongest diagnostic indicator in the case base – it cleanly separates the cooling-related repairs (C2, C3, C6 – all Overheating = Yes) from the others – so it receives the largest weight; noise and pressure each separate about half of the cases (and pressure is graded, so it already contributes partial credit) and receive equal, smaller weights. Weights are normalised by their sum, so the scale of the weights is irrelevant.

5.3 Retrieval for q = (Noise = Yes, Overheating = Yes, Pressure = Low)

Rank	Case	Noise	Heat	Pressure	sim N	sim O	sim P	Similarity	Solution
1	C3	Yes	Yes	High	1	1	0	0.700	Reduce load and inspect bearing
2	C1	Yes	No	Low	1	0	1	0.600	Replace seal
3	C2	No	Yes	Normal	0	1	0.5	0.550	Clean cooling fins
4	C5	Yes	No	Normal	1	0	0.5	0.450	Tighten mounting
5	C6	No	Yes	High	0	1	0	0.400	Replace coolant pump
6	C4	No	No	Low	0	0	1	0.300	Refill lubricant

Example: Sim(q, C3) = (0.3·1 + 0.4·1 + 0.3·0)/1.0 = **0.70**; Sim(q, C1) = (0.3·1 + 0.4·0 + 0.3·1) = **0.60**.

Proposed solution (reuse): the best match is **C3 – "Reduce load and inspect bearing"** (sim 0.70). The match is not perfect: C3 had High pressure, the new problem has Low pressure.

Revision: reducing load is intended to relieve a high-pressure condition; with Low pressure it is not needed, so the revised solution is **"Inspect bearing (pressure is Low, so load reduction is not required)"**. If the inspection and repair then succeed (confirmed by the engineer), it is **retained** as a new case:

Case	Noise	Overheating	Pressure	Solution
C7	Yes	Yes	Low	Inspect bearing (pressure is Low, so load reduction is not required)

After retention the same query retrieves C7 with similarity 1.00, followed by C3 (0.70) and C1 (0.60).

5.4 Implementation (retrieval and retention)

```
ORD = {"Low": 0, "Normal": 1, "High": 2}
W = dict(noise=0.3, heat=0.4, press=0.3)
```

```
def sim(q, c, w):
    s = dict(noise=float(q["noise"] == c["noise"]), heat=float(q["heat"] == c["heat"]),
             press=1 - abs(ORD[q["press"]] - ORD[c["press"]]) / 2)
    return sum(w[a]*s[a] for a in w) / sum(w.values()), s

def retrieve(q, base, w):
    r = [(c, *sim(q, c, w)) for c in base]
    return sorted(r, key=lambda t: (-t[1], t[0]["id"]))

def retain(base, q, solution, min_sim=0.999):        # forget duplicates
    top = retrieve(q, base, W)[0]
    if top[1] >= min_sim and top[0]["sol"] == solution: return base
    return base + [dict(id=f"C{len(base)+1}", **q, sol=solution)]
```

Doubling the weight of Overheating (0.3 : 0.8 : 0.3, normalised by 1.4):

Rank	Baseline (0.3 / 0.4 / 0.3)	Sim	Overheating doubled (0.3 / 0.8 / 0.3)	Sim
1	C3 – Reduce load and inspect bearing	0.700	C3 – Reduce load and inspect bearing	0.786
2	C1 – Replace seal	0.600	C2 – Clean cooling fins	0.679
3	C2 – Clean cooling fins	0.550	C6 – Replace coolant pump	0.571
4	C5 – Tighten mounting	0.450	C1 – Replace seal	0.429
5	C6 – Replace coolant pump	0.400	C5 – Tighten mounting	0.321
6	C4 – Refill lubricant	0.300	C4 – Refill lubricant	0.214

C3 stays first (0.786), but the order of the rest changes: the Overheating = Yes cases C2 and C6 move up to ranks 2 and 3, and C1 (a Noise/Pressure match with no overheating) drops from rank 2 to rank 4. The retrieved top-1 solution is unchanged, but a user shown the top three would now see cooling-related repairs (clean fins, replace coolant pump) instead of “replace seal”. With equal weights (1 : 1 : 1) C1 and C3 would tie at 0.667, which shows that the weights are what resolve ambiguity here and must be justified and validated with an expert.

5.5 Analysis inquiry – growth of the case base

- **Retrieval cost** grows linearly with the number of cases (every case is compared with the query: O(N·A) for A attributes) unless an index is used (case-retrieval nets, k-d trees, attribute-based indexing, pre-filtering on key attributes).
- **Case quality** can improve with coverage, but **redundant** cases (near-duplicates) add cost without information and bias the ranking towards frequent situations; **conflicting** cases (same problem description, different solutions) make the retrieved solution unreliable and can leave the “best match” arbitrary; **obsolete** cases (old equipment, outdated procedures) mislead.
- **Maintenance policy:** (1) on retain, check the nearest existing case – if similarity ≥ threshold and the solution is the same, do not store it (forget duplicates; my retain already does this); (2) if similarity ≥ threshold but solutions differ, flag for expert review and either merge or add a distinguishing attribute; (3) track each case’s usage and success rate and periodically remove unused or consistently unsuccessful cases (forgetting by utility/age); (4) occasionally re-validate the weights on held-out cases.

Q6 – Lazy versus Eager Learning under Cost and Noise (6 marks)

6.1 Choice of models

Instance-based: k-NN (k = 9, standardised features, brute force; my from-scratch rule, vectorised) – chosen because it is the Q2 model, has zero training cost and exposes the storage/latency issue directly. **Eager: a small neural network** (scikit-learn MLP, one hidden layer of 16 ReLU units, Adam, ≤ 400 epochs) – chosen because it can represent the non-linear class layout of the Q2 data (two clusters per class) which logistic regression could not, while keeping a tiny fixed parameter count (65 numbers).

Data: the Q2 generator (same features and scales), larger pool of 7,000 points per seed s ∈ {0, 1, 2} (seeds 100–102); the first 5,000 are the training pool (training sets are nested prefixes of 200–5,000), rows 5,000–5,999 are a fixed 1,000-point validation set; the test rows are not used here. (The 600-row Q2 partition is too small for sizes up to 5,000, so the same procedure is applied to a larger sample.)

6.2 Predictions made before experimenting (n from 200 to 5,000)

Quantity	k-NN (lazy)	MLP (eager)
Training time	≈ 0 (scaler + storage), essentially independent of n	grows roughly linearly with n (more gradient steps per epoch)
Prediction time / 1000 queries	grows ~linearly with n (n distances per query)	constant (fixed network)
Memory	∝ n (stores every instance)	constant (65 numbers)
Accuracy	rises with n, approaching the Bayes rate	rises then saturates early; limited by model capacity

6.3 Scaling experiment (mean of 3 seeds; accuracy ± sd across seeds)

n	Train time (k-NN / MLP)	Predict ms per 1000 (k-NN / MLP)	Stored numbers (k-NN / MLP)	k-NN val. acc.	MLP val. acc.
200	0.08 ms / 0.08 s	3.7 / 0.32	600 / 65	0.920 ± 0.028	0.916 ± 0.025
500	0.11 ms / 0.15 s	7.2 / 0.27	1500 / 65	0.927 ± 0.025	0.918 ± 0.019
1000	0.14 ms / 0.17 s	10.8 / 0.27	3000 / 65	0.935 ± 0.018	0.922 ± 0.019
2500	0.19 ms / 0.60 s	24.5 / 0.30	7500 / 65	0.938 ± 0.017	0.928 ± 0.011
5000	0.43 ms / 1.19 s	52.1 / 0.57	15000 / 65	0.942 ± 0.014	0.935 ± 0.010

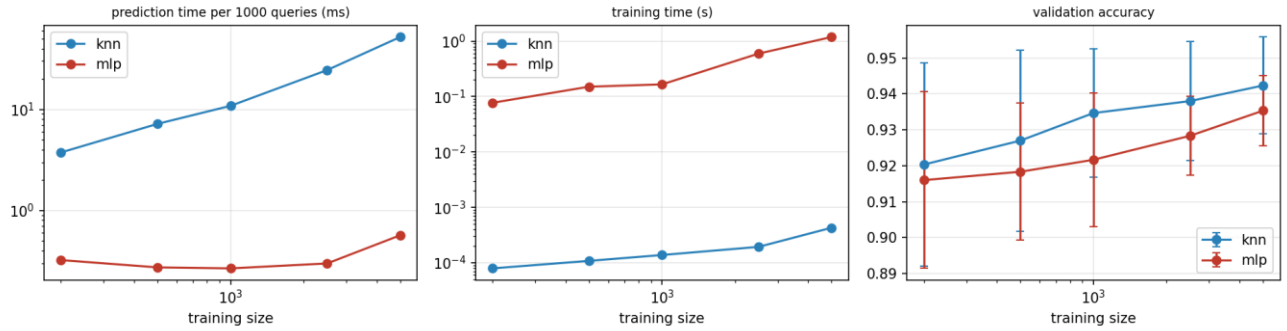


Figure 6.1 – Prediction time, training time and validation accuracy against training-set size.

The predictions are confirmed. From n = 200 to 5,000 (25×) the k-NN prediction time rises from 3.7 to 52.1 ms per 1,000 queries (≈ 14×; sub-linear at small n because of fixed overhead) and its memory grows 25×, while the MLP's prediction time stays ≈ 0.3–0.6 ms and its memory at 65 numbers. The MLP's training time rises from 0.08 s to 1.2 s; k-NN needs only ≈ 0.1–0.4 ms. Accuracy rises slowly for both (k-NN 0.920 → 0.942, MLP 0.916 → 0.935); the gap of ≈ 0.7–1.2 points is smaller than the seed-to-seed spread (1–3 points), so accuracy does not separate the models on this data.

6.4 Label noise (n = 2,000, 3 seeds, labels flipped at random with fixed seeds)

Flipped	k-NN, k = 9	k-NN, k = 25	MLP (16 hidden)
0%	0.940 ± 0.015	0.937 ± 0.017	0.925 ± 0.015
10%	0.937 ± 0.016	0.936 ± 0.020	0.924 ± 0.017
20%	0.917 ± 0.023	0.937 ± 0.021	0.919 ± 0.018

Validation accuracy is measured on clean labels. At 20% noise k-NN with the fixed k = 9 loses 2.3 points (0.940 → 0.917), because each noisy training label directly affects the vote of nearby queries; the MLP loses only 0.7 points (0.925 → 0.919): it averages noise across all points when fitting a smooth function. A larger k (25) makes k-NN as robust as the MLP (0.937 at 0%, 0.937 at 20%) – k acts as the noise-smoothing parameter – but k should then be re-selected under noise, which requires clean validation labels. 1-NN would be the most fragile.

6.5 Brute force versus tree index (n = 5,000 training, 1,000 queries, k = 9)

Dimensions	Brute (ms)	kd-tree (ms)	Ball tree (ms)	kd speed-up	Ball speed-up
2	32.8	3.4	12.0	9.54×	2.74×
5	16.6	7.7	21.1	2.16×	0.79×
10	17.6	50.9	40.9	0.35×	0.43×
20	17.2	114.6	63.5	0.15×	0.27×
50	20.7	247.0	144.5	0.08×	0.14×

Query time only (index build excluded: ≈ 2–15 ms). Features: the two informative Q2 features plus (d − 2) independent N(0,1) features, standardised. Library indexes (scikit-learn NearestNeighbors) are used here as the tree structures; single timing run on one CPU core, so values are indicative.

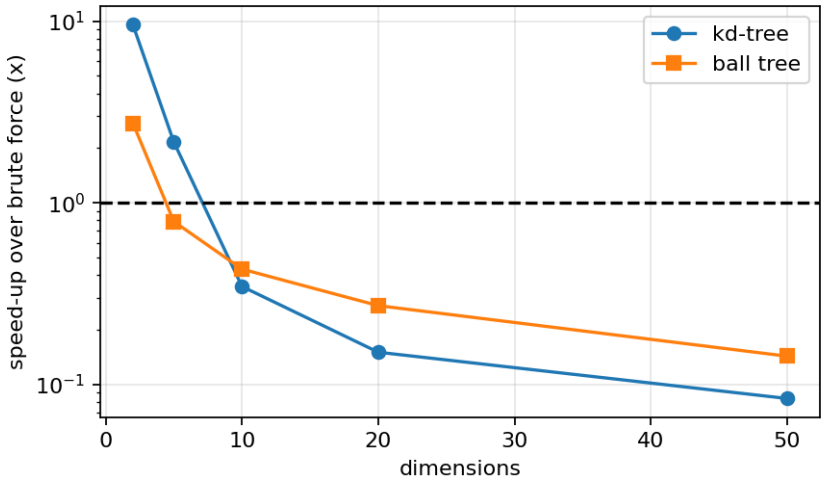


Figure 6.2 – Tree-index speed-up over brute force (dashed line = break-even).

At d = 2 the kd-tree is ≈ 9.5× faster than brute force and at d = 5 still ≈ 2.2×; the speed-up **disappears between d = 5 and d = 10** (0.35× at d = 10) and the trees are 7–12× slower at d = 50. This is the same distance-concentration effect as in Q2: when nearest and farthest neighbours are almost equally distant, the bounding regions cannot be pruned, the tree visits almost every node and adds overhead. The ball tree is less effective than the kd-tree even at low d (0.79× at d = 5) and also loses at d ≥ 10.

6.6 Recommendation and limits

Recommendation for the streaming deployment: the eager MLP. On this problem it matches k-NN accuracy (within ≈ 1 point), predicts 10–90× faster (0.3–0.6 ms vs 3.7–52 ms per 1,000 queries, and the gap grows with n), needs 65 stored numbers instead of up to 15,000, and degrades least under label noise at fixed hyper-parameters. Its training time (≈ 1 s at n = 5,000) is paid once and is negligible against the deployment horizon.

The recommendation would change if: (1) new labelled cases must be usable immediately, or the concept drifts, so that cheap incremental updating (adding/removing instances) matters more than latency; (2) the training set is small (a few hundred points) and prediction rate is low – k-NN has no training cost and equal accuracy; (3) the class boundary is far more irregular than a 16-unit network can follow, or interpretability by example ("this machine resembles these three") is required; (4) the data are low-dimensional ($d \leq 5$) and a kd-tree can bring k-NN latency down; (5) labels are very noisy and a large k (or a tree-pruned/condensed k-NN) can be validated on clean data.

Limits of the evidence: three seeds (standard deviations of 1–3 points mean sub-1-point differences are not significant); sizes only up to 5,000 and a single 2-D synthetic problem with a fixed 1,000-point validation set; noise is symmetric, uniformly random label flipping only (real label noise is usually class- or region-dependent); one MLP architecture with default optimiser settings (not tuned); timings from one CPU core with Python/NumPy overheads, the brute-force k-NN being my vectorised version and the tree indexes being library code; memory is counted as stored numbers, not actual RAM.

Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate (13 marks)

Selected solution: the standardised k-NN classifier of Q2 (k = 9, uniform vote), whose significant limitation is its **degradation in the presence of irrelevant features (high-dimensional distances)**.

A. Diagnosis (2 marks)

Total features	Val. accuracy	d_nearest / d_farthest
2	0.940 ± 0.000	0.016
4	0.902 ± 0.014	0.083
7	0.873 ± 0.021	0.188
12	0.839 ± 0.015	0.306
22	0.761 ± 0.014	0.429
52	0.707 ± 0.048	0.588

Original solution, 5 noise seeds (11–15), k = 9; the same degradation curve as Fig. 2.2.

Accuracy falls from 0.940 to 0.707 (−23 points) as 50 irrelevant features are added, while the contrast ratio rises from 0.016 to 0.589: nearest and farthest neighbours become almost equally distant, so the "nearest" neighbours are chosen by noise. The bottleneck is therefore the **distance measure / feature representation** – an unweighted Euclidean distance that treats all coordinates as equally relevant – and not the other candidates:

- **Not the choice of k:** re-tuning k at 52 features (mean of 5 seeds) gives 0.619 (k = 1), 0.707 (k = 9), 0.730 (k = 25) and at best 0.782 (k = 51) – still 16 points below the 2-feature accuracy – whereas a very large k costs accuracy when there is no noise (k = 51: 0.905 vs 0.940 at k = 9).
- **Not data sparsity or label noise:** the same 600 training points give 0.94 with the 2 informative features only. **Not computational cost:** the issue is accuracy, not latency (although the extra 50 dimensions also make every distance 26× longer to compute).

B. Design objective (2 marks)

Item	Statement
Baseline	Standardised k-NN, k = 9: validation accuracy 0.707 (mean of 5 seeds) at 52 features (2 informative + 50 irrelevant); 0.940 at 2 features.
Target	Validation accuracy ≥ 0.92 (within 2 points of the 2-feature accuracy) at every tested number of irrelevant features $p \in \{0, 2, 5, 10, 20, 50\}$, i.e. accuracy that is stable in p; nearest/farthest ratio no higher than the 2-feature value.
Acceptable trade-offs	Extra one-off cost at training (feature search) up to ≈ 1 s for n = 600; prediction time per query not higher than the baseline (ideally lower); labels required for the selection; use of training data only (no validation/test leakage).

C. Revised learning strategy (4 marks)

What changes and why. The original distance gives each coordinate weight 1. The redesign makes the distance **supervised**: it learns, from the training set alone, which features matter and removes the others, so that the distance is computed only in the relevant subspace. This addresses the diagnosed property – label-independent coordinates adding random variance to every distance.

Version 1 (first hypothesis): relevance-weighted distance with pruning. Weights $w_j = MI(x_j; y)/\max_i MI(x_i; y)$ (mutual information estimated on the training set), distance $d^2 = \Sigma\, w_j\, (x_j - q_j)^2$, and features with $w_j < 0.25$ are pruned. It is cheap (one pass over the features) and "soft".

Version 2 (final design after Version 1 failed – see F): leave-one-out forward feature selection (wrapper). The relevance of a feature subset is measured by the leave-one-out (LOO) accuracy of k-NN on the training set itself;

features are added greedily while LOO accuracy improves by at least 0.5 points. Test and validation sets are never used for selection.

```
FORWARD-SELECT-kNN(Xtr_std, ytr, k=9, min_gain=0.005)
 S <- {} ; best <- 0
 repeat
    for each feature j not in S:
       acc[j] <- LOO-accuracy of k-NN on Xtr_std[:, S ∪ {j}]  (exclude each point from its own neighbour list)
    j* <- argmax acc ;  gain <- acc[j*] - best
    if gain < min_gain: stop
    S <- S ∪ {j*} ; best <- acc[j*]
 return S
PREDICT(q):  standardise q with training statistics; keep only columns in S; k-NN vote (k=9) on Xtr_std[:, S]
```

Workflow: standardise (train statistics) → forward selection by training-LOO accuracy → restrict training and query vectors to S → ordinary k-NN. **Extra cost:** the search needs O(|S|·F) distance-matrix evaluations of size n × n (≈ 0.5 s for F = 52, n = 600 here) and n^2 memory during the search; it is paid once. **Savings:** every prediction uses |S| instead of F coordinates. **New limitations:** greedy search can miss features that are useful only jointly with others (a single feature must give a first gain); LOO-based selection can overfit when F is large relative to n; it needs labels; selected features are fixed and must be re-selected under concept drift; the stopping gain (0.5 points) is a hyper-parameter.

D. Prediction before implementing (1 mark)

Prediction (from the diagnosis): removing the label-independent coordinates restores the distance geometry of the 2-feature problem, so for the redesigned solution (i) validation accuracy should stay ≈ 0.94 (±2 points) for all p from 0 to 50, i.e. **flat in p**; (ii) the nearest/farthest ratio should stay at its 2-feature value (≈ 0.016) instead of rising to 0.59; (iii) prediction time at 52 features should fall roughly in proportion to the number of retained coordinates; (iv) the main risk is wrongly discarding a weak informative feature, which would cap accuracy below 0.94. The measure expected to improve is **validation accuracy at 52 features: 0.71 → ≈ 0.94**.

E. Implementation and validation (2 marks)

Same data partition (seed 42, 600/200/200), same k = 9, same five noise seeds (11–15) and the same standardisation as the baseline; prototypes in Python/NumPy, mutual information from scikit-learn (Version 1 only).

Irrelevant features p	Original acc. ± sd	V1 (MI + prune) acc. ± sd	V2 (LOO selection) acc. ± sd	Features kept V1 / V2	Both informative kept V1 / V2	Ratio d_nn/d_far V2
0	0.940 ± 0.000	0.915 ± 0.000	0.940 ± 0.000	1 / 2	0% / 100%	0.016
2	0.902 ± 0.014	0.915 ± 0.000	0.940 ± 0.000	1 / 2	0% / 100%	0.016
5	0.873 ± 0.021	0.915 ± 0.000	0.940 ± 0.000	1 / 2	0% / 100%	0.016
10	0.839 ± 0.015	0.915 ± 0.000	0.940 ± 0.000	1 / 2	0% / 100%	0.016
20	0.761 ± 0.014	0.915 ± 0.000	0.940 ± 0.000	1 / 2	0% / 100%	0.016
50	0.707 ± 0.048	0.915 ± 0.000	0.940 ± 0.000	1 / 2	0% / 100%	0.016

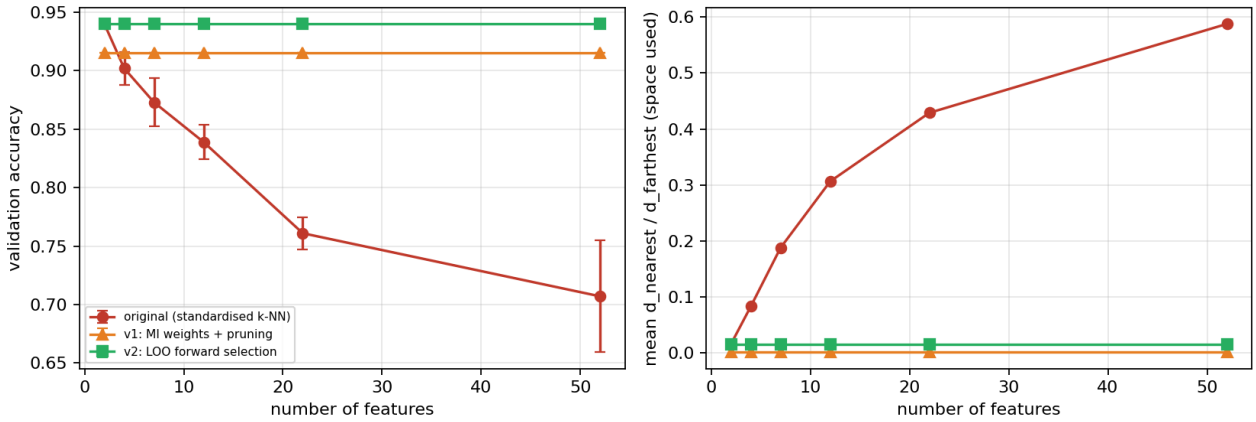


Figure 7.1 – Validation accuracy and distance contrast against number of features for the original k-NN and the two redesigns.

Measure at 52 features (200 validation queries)	Original	Redesign V2
Prediction time (ms, my k-NN implementation)	47.1	17.6 (2.7× faster)
One-off selection cost (ms)	0	470
Coordinates used per distance	52	2
Stored numbers (600 points)	31,200	1,200

Single test evaluation (the final design is fixed; train + validation refit; a fresh draw of 50 irrelevant features; selected features $\{x_2, x_1\}$): test accuracy **0.930** for the redesign against **0.710** for the original (the Q2 2-feature test accuracy was 0.93).

F. Justification of the final design (2 marks)

- **Objective achieved by V2:** validation accuracy 0.940 at 52 features (baseline 0.707, +23.3 points; target ≥ 0.92), flat in p with zero variance across noise seeds (the same two features were selected in every run), contrast ratio unchanged (0.016), 2.7× faster prediction, and a 26× smaller stored data set; test accuracy 0.93 vs 0.71. The prediction in D is confirmed.
- **Version 1 failed its hypothesis, and the failure is informative.** It made accuracy stable in p (0.915 at every p) but 2.5 points below the 2-feature accuracy, and below the target of 0.92. The marginal MI scores of the two informative features were 1.00 (x_2, the ratio) and only 0.10 (x_1, the count); with the 0.25 cut-off V1 always discarded x_1 (both informative features were retained in 0% of runs), keeping a single feature. The data have two clusters per class, so x_1 is informative only in combination with x_2; a univariate score cannot see this. The wrapper (V2) evaluates feature subsets jointly with the actual classifier and recovers both features in 100% of runs. A soft-weighting variant without pruning (same experiment) gave 0.89–0.915 at all p – far better than the original at large p but never reaching 0.94, again because x_1 is down-weighted.
- **New costs:** a one-off ≈ 0.5 s search and n^2 memory during it; need for labels; selection is greedy and based on LOO accuracy; the redesigned model is only as good as its subset (a feature that is useful only in an interaction with no individual gain would be missed). Plain k-NN remains preferable when all features are known to be relevant (here at p = 0 the two methods are identical, 0.94, so the search is wasted effort), when n is too small for stable LOO estimates, or when the feature set is so large that the O(F) search per step becomes prohibitive (then a filter method or PCA/LDA is used first).
- **Limits of this validation:** one dataset family with Gaussian, label-independent irrelevant features (the easiest case for selection – correlated or weakly informative distractors would be harder); k fixed at 9; the 200-point validation set gives a standard error of ≈ 1.7 points; the five noise seeds vary only the irrelevant features, not the data split; the test comparison is a single 200-point evaluation.

Final design decision: adopt standardised k-NN with LOO forward feature selection (V2) for deployment on feature sets that may contain irrelevant columns; keep V1 only as a documented negative result.